此次练习中，我们使用Human Activity Recognition Using Smartphones数据集。它通过对参加测试者的智能手机上安装一个传感器而采集了参加测试者每天的日常活动（ADL）。目标是将日常活动分成六类（walking， walking upstairs， walk downstairs， sitting， stand， and laying）。

该数据集也可以在Kaggle网站上获得：https://www.kaggle.com/uciml/human-activity-recognition-with-smartphones/downloads/human-activity-recognition-with-smartphones.zip

把训练文件重新命名为 Human_Activity_Recognition_Using_Smartphones_Data.csv

第一步：导入数据

- 查看数据类型---因为有太多的列，所以最好使用value_counts
- 判断其中的小数数值是否需要尺度缩放
- 检查数据中各活动类型的划分
- 把活动类型标签编码成一个整数

在 [2] …

```
import pandas as pd
import numpy as np

filepath = 'Human_Activity_Recognition_Using_Smartphones_Data.csv'
data = pd.read_csv(filepath)
```

所有列的数据类型都是浮点数，除了活动标签列。

在 [3] …

```
data.dtypes.value_counts()
```

输出[3]:

```
float64    561
int64        1
object       1
Name: count, dtype: int64
```

在 [4] …

```
data.dtypes.tail()
```

输出[4]:

```
angle(X,gravityMean)    float64
angle(Y,gravityMean)    float64
angle(Z,gravityMean)    float64
subject                   int64
Activity                 object
dtype: object
```

数据都已经全部被缩放到-1到1之间了。

在 [5] …

```
data.iloc[:, :-1].min().value_counts()
```

出[5]:

```
-1.000000    466
-0.995377      2
-0.999996      2
-0.999893      2
-1.000000      2
            ...
-0.999983      1
-0.943439      1
-0.998014      1
-0.999915      1
 1.000000      1
Name: count, Length: 93, dtype: int64
```

在 [6] …

```
data.iloc[:, :-1].max().value_counts()
```

出局[6]:

```
1.000000     452
0.994731       2
0.805064       1
0.908361       1
0.891736       1
            ...
0.990935       1
0.979031       1
0.928416       1
0.848031       1
30.000000      1
Name: count, Length: 110, dtype: int64
```

检查数据中各活动类型的划分---已经比较平衡了。

在 [7] …

```
data.Activity.value_counts()
```

出局[7]:

```
Activity
LAYING                1407
STANDING              1374
SITTING               1286
WALKING               1226
WALKING_UPSTAIRS      1073
WALKING_DOWNSTAIRS     986
Name: count, dtype: int64
```

Scikit learn的分类器不接受一个稀疏矩阵作为预测列。所以，可以使用将活动标签编码为整数。 `LabelEncoder`

在 [8] …

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
data['Activity'] = le.fit_transform(data.Activity)
data['Activity'].sample(5)
```

出局[8]:

```
4696    2
1335    2
6765    4
578     0
2477    3
Name: Activity, dtype: int32
```

第二步：划分训练数据和测试数据

- 可以考虑使用Scikit-learn中的，以保证划分后的数据集中每个类别个案的比例与整个数据集相同。 `StratifiedShuffleSplit`

In [9]:
```
feature_cols = data.columns[:-1]

from sklearn.model_selection import StratifiedShuffleSplit

# Get the split indexes
strat_shuf_split = StratifiedShuffleSplit(n_splits=1,test_size=0.3, random_state

train_idx, test_idx = next(strat_shuf_split.split(data[feature_cols], data.Activ

# Create the dataframes
X_train = data.loc[train_idx, feature_cols]
y_train = data.loc[train_idx, 'Activity']

X_test  = data.loc[test_idx, feature_cols]
y_test  = data.loc[test_idx, 'Activity']
```

In [10]:
```
y_train.value_counts(normalize=True)
```

Out[10]:
```
Activity
0    0.191411
2    0.186941
1    0.174893
3    0.166731
5    0.145939
4    0.134085
Name: proportion, dtype: float64
```

In [11]:
```
y_test.value_counts(normalize=True)
```

Out[11]:
```
Activity
0    0.191296
2    0.186763
1    0.174977
3    0.166818
5    0.145966
4    0.134180
Name: proportion, dtype: float64
```

第三步：训练模型

- 用所有特征训练一个基本的使用缺省参数的逻辑回归模型。
- 分别用L1和L2正则化来训练一个模型，使用交叉验证确定超参数的值。注意，正则化模型，尤其是L1模型可能需要一定训练时间。

In [12]:
```
# 请在此处填写你的代码（训练一个基本的使用缺省参数的逻辑回归模型）
from sklearn.linear_model import LogisticRegression

# 训练基本的逻辑回归模型
lr = LogisticRegression(solver='liblinear').fit(X_train, y_train)
```

In [13]:
```
from sklearn.linear_model import LogisticRegressionCV
```

```
# L1 正则化的逻辑回归
lr_l1 = LogisticRegressionCV(Cs=10, cv=4, penalty='l1', solver='liblinear').fit(
```

In [14]:
```
# 请在此处填写你的代码（L2 正则化的逻辑回归）
from sklearn.linear_model import LogisticRegressionCV

# L2 正则化的逻辑回归
lr_l2 = LogisticRegressionCV(Cs=10, cv=4, penalty='l2', solver='liblinear').fit(
```

第四步：

- 输出上面训练出的三个模型中每个特征的系数；
- 并绘制成图来比较它们的差异 （每个类别一张图）

In [15]:
```
# 请在此处填写你的代码（输出各模型训练到的特征系数值）
import pandas as pd

# 基本逻辑回归模型的系数
lr_coef = pd.DataFrame(lr.coef_, columns=feature_cols)
# L1 正则化逻辑回归模型的系数
lr_l1_coef = pd.DataFrame(lr_l1.coef_, columns=feature_cols)
# L2 正则化逻辑回归模型的系数
lr_l2_coef = pd.DataFrame(lr_l2.coef_, columns=feature_cols)
```

In [15]:
```
# 请在此处填写你的代码（绘制6张图）
```

In [18]:
```
## 第五步：预测数据
# 基本逻辑回归模型预测
y_pred_lr = lr.predict(X_test)
# L1 正则化逻辑回归模型预测
y_pred_l1 = lr_l1.predict(X_test)
# L2 正则化逻辑回归模型预测
y_pred_l2 = lr_l2.predict(X_test)
```

In [19]:
```
## 第六步：训练不同参数的模型
## 修改第三步中L1、L2模型参数，尝试得到最佳结果并输出
# 尝试不同的 Cs 和 cv 参数
# L1 正则化的逻辑回归
lr_l1_new = LogisticRegressionCV(Cs=[0.1, 1, 10, 100], cv=5, penalty='l1', solve
# L2 正则化的逻辑回归
lr_l2_new = LogisticRegressionCV(Cs=[0.1, 1, 10, 100], cv=5, penalty='l2', solve
```

In [ ]:
```
## 第七步：评价模型

对第六步的2个模型，分别计算下面的各评测指标值：

* accuracy
* precision
* recall
* fscore
* confusion matrix
```

In [20]:
```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_sc

# 评价 L1 新模型
y_pred_l1_new = lr_l1_new.predict(X_test)
```

```python
accuracy_l1 = accuracy_score(y_test, y_pred_l1_new)
precision_l1 = precision_score(y_test, y_pred_l1_new, average='weighted')
recall_l1 = recall_score(y_test, y_pred_l1_new, average='weighted')
f1_l1 = f1_score(y_test, y_pred_l1_new, average='weighted')
conf_matrix_l1 = confusion_matrix(y_test, y_pred_l1_new)

print(f"L1 正则化模型 - 准确率: {accuracy_l1}")
print(f"L1 正则化模型 - 精确率: {precision_l1}")
print(f"L1 正则化模型 - 召回率: {recall_l1}")
print(f"L1 正则化模型 - F1 分数: {f1_l1}")
print("L1 正则化模型 - 混淆矩阵:")
print(conf_matrix_l1)

# 评价 L2 新模型
y_pred_l2_new = lr_l2_new.predict(X_test)
accuracy_l2 = accuracy_score(y_test, y_pred_l2_new)
precision_l2 = precision_score(y_test, y_pred_l2_new, average='weighted')
recall_l2 = recall_score(y_test, y_pred_l2_new, average='weighted')
f1_l2 = f1_score(y_test, y_pred_l2_new, average='weighted')
conf_matrix_l2 = confusion_matrix(y_test, y_pred_l2_new)

print(f"L2 正则化模型 - 准确率: {accuracy_l2}")
print(f"L2 正则化模型 - 精确率: {precision_l2}")
print(f"L2 正则化模型 - 召回率: {recall_l2}")
print(f"L2 正则化模型 - F1 分数: {f1_l2}")
print("L2 正则化模型 - 混淆矩阵:")
print(conf_matrix_l2)
```

```
L1 正则化模型 - 准确率: 0.985494106980961
L1 正则化模型 - 精确率: 0.9855013331908217
L1 正则化模型 - 召回率: 0.985494106980961
L1 正则化模型 - F1 分数: 0.9854921396229933
L1 正则化模型 - 混淆矩阵:
[[422   0   0   0   0   0]
 [  0 369  17   0   0   0]
 [  0  14 398   0   0   0]
 [  0   0   0 368   0   0]
 [  0   0   0   0 296   0]
 [  0   0   0   1   0 321]]
L2 正则化模型 - 准确率: 0.986400725294651
L2 正则化模型 - 精确率: 0.9864082252953139
L2 正则化模型 - 召回率: 0.986400725294651
L2 正则化模型 - F1 分数: 0.9863988819624415
L2 正则化模型 - 混淆矩阵:
[[422   0   0   0   0   0]
 [  0 370  16   0   0   0]
 [  0  13 399   0   0   0]
 [  0   0   0 368   0   0]
 [  0   0   0   0 296   0]
 [  0   0   0   1   0 321]]
```

In [ ]: